Baseline Summary — CTGAN / TVAE (Deep Generative Tabular)

Fourth baseline: neural generative models for the three session variables (arrival hour, plug-in duration, energy). Xu et al., "Modeling Tabular Data using Conditional GAN", NeurIPS 2019. Code: github.com/sdv-dev/CTGAN (the `ctgan` package).

1. What they are

CTGAN and TVAE are the two models introduced in the same NeurIPS 2019 paper for generating synthetic *tabular* data. Both learn the joint distribution of the columns and sample new rows; they differ in the generative machinery:

- **CTGAN** — a conditional generative adversarial network. A generator proposes fake rows and a discriminator tries to tell them from real; training is the usual adversarial min-max. Key tabular tricks: *mode-specific normalisation* (each continuous column is modelled as a Gaussian mixture and encoded relative to its mode, handling multimodal/skewed columns) and *training-by-sampling* with a conditional vector to balance rare categories.
- **TVAE** — a variational autoencoder for the same tabular encoding: an encoder maps a row to a latent Gaussian, a decoder reconstructs it; trained by maximising the evidence lower bound (ELBO). Generation samples the latent prior and decodes. Typically more stable to train than a GAN, and it has direct EV precedent (VAE for EV load profiles, Energies 2019).

2. How it is used as a generator

- Build a 3-column table (arrival with a 6 AM origin to avoid the midnight wrap, duration, energy); all continuous.
- Fit the model (`CTGAN(epochs=...).fit(df)` or `TVAE(...)`); mode-specific normalisation handles the different scales.
- Sample N synthetic rows, invert the arrival shift, write `results/{ctgan,tvae}_<dataset>.csv` for the shared comparison harness.

Implemented in `benchmarking/ctgan/run_ctgan.py` (`--model ctgan|tvae --epochs N`), same interface as the other baselines.

3. Strengths, limitations, and a compute caveat

Strengths	Limitations
Data-fitted deep generative model; the recognised "modern ML" comparison reviewers expect.	Static i.i.d. sessions: no per-user identity, no idle-gap / sequential structure.
Mode-specific normalisation handles skewed, multimodal columns.	TVAE (a VAE) tends to *over-smooth heavy tails* — it under-models the long-duration residential sessions.
CTGAN available in the same script for the GAN variant.	Training is compute-heavy: a fair fit needs ~300 epochs on the full data (GPU-preferred).

4. Role in our study & how to obtain the final number

CTGAN/TVAE is the deep-generative member of the baseline suite (EV-SDG = parametric, GMMNet = conditional-density neural, CTGAN/TVAE = GAN/VAE). Compute-limited sandbox runs (small subsample, capped epochs) place TVAE well behind GMMNet on marginals, dominated by the duration tail it smooths over — consistent with the known VAE behaviour, but those runs are a lower bound, not the final figure. For the paper, train it properly:

```
python benchmarking/ctgan/run_ctgan.py combined --model tvae --epochs 300
python benchmarking/ctgan/run_ctgan.py combined --model ctgan --epochs 300
```

then re-run `compare.py` / `metrics.py`. Expected outcome: competitive with GMMNet on marginals, similarly unable to reproduce per-user heterogeneity or the idle gap — so it reinforces, rather than threatens, the paper's core claim.